

STREAM: Spatiotemporal Similarity-based Efficient Approximate Median with Tunable Granularity

Fenfang Li^{1,2}, Huizhang Luo¹, Weichen Liu², Anthony Theodore Chronopoulos³, Kenli Li¹, Chubo Liu¹

¹Hunan University, China; ²Nanyang Technological University, Singapore; ³The University of Texas at San Antonio, USA

Abstract—The median (MED) is a crucial statistic for measuring the central tendency. However, exact MED computation remains costly, with even state-of-the-art (SOTA) algorithms failing to meet (near) real-time processing demands. While approximate MED algorithm has arisen as a promising candidate, existing approaches ignore the potential opportunity of spatiotemporal similarity within the application and fail to provide application-specific trade-offs between execution time and accuracy. Our goal is to design an enhanced approximate MED algorithm STREAM, which is capable of exploiting the spatiotemporal similarity to achieve bucket reuse and establish a tunable-grained bucket mechanism to meet the accuracy of application-specific requirements. Experimental results show that while maintaining nearly identical accuracy, STREAM outperforms the SOTA approximate methods DDSketch (up to 10 $\times$, 4.7 $\times$ on average) and KLL (up to 71.2 $\times$, 10.1 $\times$ on average).

Index Terms—approximate computing, approximate median, spatiotemporal similarity, synopsis.

I. INTRODUCTION

The identification of significant elements within large datasets is a fundamental challenge that drives numerous fields, such as embedded system libraries [1]–[4], computer vision [5], and data mining [6], [7]. Among them, median (MED) calculation emerges as a crucial special case, remaining resilient to extreme values and providing unique insights into the central tendency of the dataset.

Unfortunately, computing the exact MED over a set of n elements becomes prohibitive for embedded systems requiring real-time or near real-time responses. Compute-intensive applications, such as real-time monitoring, streaming data, and image processing, often operate under strict resource constraints in embedded systems. The high time complexity of exact MED algorithms, typically $O(n\log n)$ for unsorted data, can lead to significant processing delays, impeding the response essential to these applications. Moreover, conventional exact MED algorithms depend on global or partial sorting steps, which can cause further latency. Therefore, a new design paradigm—*approximate computing* is considered a more feasible solution, which increases performance by controllably reducing accuracy [8]–[11].

Recently, popular approximate median (AM) algorithms for streaming data processing can be categorized into two classes: *bucket-based algorithms* and *compactor-based algorithms*. Bucket-based algorithms, such as DDSketch [12] and HDRHistogram [13], approximate the MED by allocating data into fixed-size buckets and maintaining a compact synopsis of the data distribution. Compactor-based algorithms,

including KLL [14] and KLL $\pm$ [15], rely on layered sampling and compressed representation to efficiently estimate the approximate MED. While effective in some applications, one significant limitation is that these algorithms ignore the spatiotemporal similarity (STS) commonly present in real-time applications [16], where consecutive data segments often exhibit similarity distribution. Specially, they treat each data segment as independent input and compute approximate MED completely from scratch without reusing information from the previous data segments. Thus, this leads to redundant computation in contexts where STS exists.

Another important limitation lies in the limited adaptability of existing algorithms for diverse application needs. The embedded system applications mentioned earlier are inherently error-tolerant, i.e., they generally require acceptable results rather than fully exact [11]. For example, a few pixels drop of processed image are imperceptible to human eyes. However, diverse applications exhibit distinct trade-offs between computational efficiency and accuracy. Limited research has explored tunable mechanisms to adjust approximation strategies based on application-specific requirements, leaving a gap in the current research field.

We propose a novel, simple yet effective algorithm, STREAM. It exploits the STS in application to improve performance. Unlike existing algorithms that process each data segment independently, STREAM allows subsequent data segments to reuse buckets of previous data segments. STREAM integrates bucketing and statistical metric computation of current data segment in a single pass, reducing execution time. Moreover, STREAM provides a tunable option, allowing tunable granularity in bucket strategies (coarse/fine-grained level), to achieve a trade-off between speed and accuracy for diverse applications. In particular, STREAM-CG (coarse-grained) constructs the buckets based on the 68-95-99.7 rule and allocates data into fixed-width buckets, achieving rapid, storage-light MED estimation suited for real-time processing. To enhance accuracy, STREAM-FG (fine-grained) employs a bucket tuning strategy that narrows the boundaries using a scale function. This solution places finer buckets around the region closer to the exact MED, in contrast to the fixed-width bucket of STREAM-CG, thereby achieving higher accuracy.

In summary, this paper makes the following contributions:

- We propose STREAM, a novel bucket-based framework that exploits the potential STS in applications. It computes current approximate MED by reusing buckets of previous data segments if STS exists between consecutive

Corresponding authors: luohuizhang@hnu.edu.cn, liuchubo@hnu.edu.cn

data segments.

- We propose a tunable mechanism to adjust bucket strategy for diverse applications. STREAM-CG provides a coarse-grained implementation, which meets the requirements of applications that tolerate some error with mild perceived effect to obtain significant speed improvement. STREAM-FG provides a fine-grained implementation, which refines the median estimate by concentrating the buckets in regions closer to the exact MED.
- Experimental results show that while maintaining nearly identical accuracy, STREAM outperforms the SOTA approximate methods DDSketch (up to 10x, 4.7x on average) and KLL (up to 71.2x, 10.1x on average).

II. BACKGROUND AND MOTIVATION

A. Approximate Computing

Approximate computing (AxC) is an emerging paradigm for energy-efficient and/or high-performance design [11]. Companies, such as IBM, Google, Intel, and ARM, are actively engaged in pioneering research and developing commercial offerings that incorporate AxC strategies. Generally, AxC techniques are categorized into six classes: (i) *selective task skipping*, (ii) *approximate memoization*, (iii) *relaxed synchronization*, (iv) *precision scaling*, (v) *data sampling*, and (vi) *approximate programming languages*. AxC can provide effective trade-offs between execution time, accuracy, power, and so on, making it a powerful tool for applications with diverse performance needs.

- **Need for Low-Power/Energy Computing.** With transistor scaling reaching deep nanometer domains, power and energy consumption have become the primary constraints in modern computing systems. In this context, AxC can effectively balance power consumption with accuracy, area, and other performance requirements of applications.
- **Need for Accelerated Computing.** With the explosive growth in data volume, compute-intensive workloads of applications challenge their efficient implementation in terms of speed. AxC addresses this by enabling effective trade-offs between speed and desired accuracy.

Among various approximate computing techniques, approximate MED computation becomes a research hotspot, which can address the challenges faced by resource constraints and provide faster response times and comparable accuracy for compute-intensive applications [17]. However, existing approximate MED algorithms fail to use potential STS presenting in streaming data processing and lack the tunability to adapt to diverse application requirements.

B. Limitations of Approximate Median Calculation

Unexploited Spatiotemporal Similarity. Existing approximate MED algorithms fail to utilize the potential STS between consecutive data segments. To gain insight into STS, we observe the distribution within an application. As shown in Fig. 1, consecutive segments exhibit high similarity, as observed from their overlapping distribution curves. This overlap reflects comparable statistical properties, such as mean and

standard deviation, and also validates the premise of reusing bucket partitions across segments (i.e., exploit the properties of $\mathbb{S}_i$ to optimize the computation of $\mathbb{S}_{i+1}$). To further investigate the statistical significance of differences between distributions, we also employ the Mann Whitney U (MWU)¹ test, a non-parametric statistical technique that evaluates the central tendencies of the two distributions (i.e., their medians). As shown in Fig. 2, all p-values of consecutive segments exceed the significance level of alpha 0.05, indicating consecutive segments show no statistically significant median differences. Thus, medians of consecutive segments fall into similar buckets, enabling reduced execution time by reusing buckets from previous data segments.

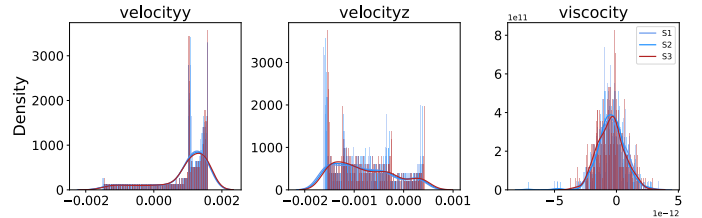


Fig. 1: Distribution of three consecutive data segments ($\mathbb{S}_1$, $\mathbb{S}_2$, $\mathbb{S}_3$) in application *Miranda*². Overlaid on this scatter plot are density curves, fitted with kernel density estimation. $\mathbb{S}_i$ represents the i -th data segment.

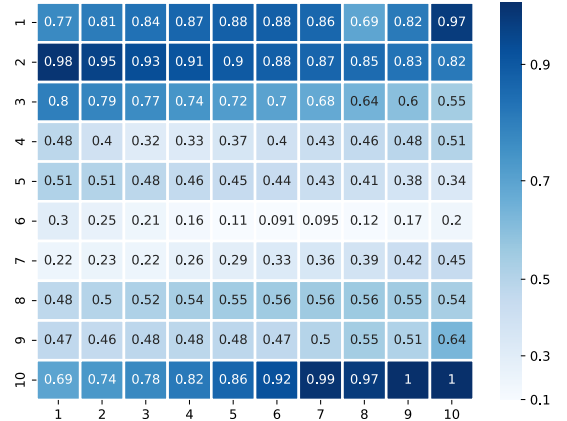


Fig. 2: Heatmap of Mann Whitney U test for consecutive streaming data segments ($\mathbb{S}_1$ to $\mathbb{S}_{101}$). Each cell represents the p-value ($\alpha = 0.05$) between consecutive segments ($\mathbb{S}_i$, $\mathbb{S}_{i+1}$). Darker color indicates higher values.

Limited Adaptability for Application-Specific Needs. Existing bucketing strategies generally adopt a one-size-fits-all approach, which limits their adaptability across various applications with varying requirements for speed and accuracy. Table I compares different applications, highlighting their trade-offs. Power monitoring and image processing often prioritize processing speed over exact accuracy, whereas healthy wearable devices and sensors require high accuracy to ensure the reliability of measurements. For applications with high accuracy requirements, only coarse-grained bucket strategies

¹https://en.wikipedia.org/wiki/Mann-Whitney_U_test

²<https://sdrbench.github.io/>

TABLE I: Requirements of accuracy and speed for different applications [11].

Applications	Accuracy	Speed
Environmental monitoring	Medium	Medium
Power monitoring	Midum	High
Image processing	Medium to High	High
Health wearable devices	High	Medium to High
Disaster warning sensor	High	High

may lead to significant errors due to unreasonable boundaries. On the contrary, for applications with low accuracy requirements, only fine-grained strategies can result in a waste of computational resources to obtain a fined boundary. Thus, a tunable strategy can more effectively balance speed and accuracy, enhancing adaptability for application-specific.

III. DESIGN AND IMPLEMENTATION

A. STREAM Framework

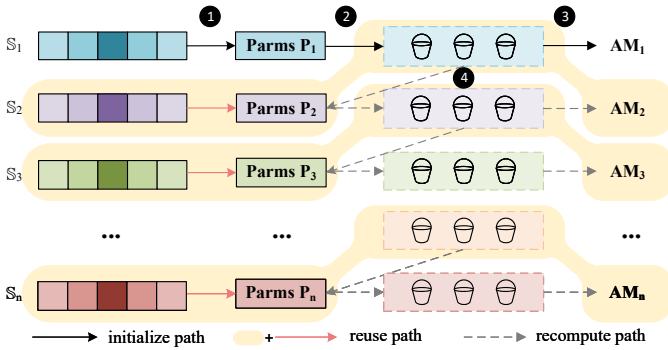


Fig. 3: The framework using STS to get approximate MED, where initialize path follows steps ①+②+③, reuse path follows steps ② (①)+ ③, and recompute path follows steps ④+③.

Fig. 3 illustrates the framework of STREAM that calculates the approximate MED with STS. For the initial streaming data segment S_1 , STREAM follows the initialize path: ① traversing the segment to determine the bucket partitioning parameters P_1 , ② re-traversing and allocating data into buckets based on P_1 , ③ finding the bucket holding MED and identifying the approximate MED AM_1 using the bucket's boundaries. For the subsequent streaming data segment S_2 , STREAM utilizes the STS to reuse the bucket: STREAM first traverses S_2 , while simultaneously performing two operations: allocating data into buckets based on the bucket partitions from S_1 and computing the current bucket parameters P_2 (Step ② (①)). Then, two scenarios may occur: 1) If STS is confirmed, STREAM directly identifies the MED bucket and computes AM_2 using its boundaries (Step ③). 2) If similarity validation fails, STREAM executes recompute step ④, which requires bucketing with P_2 before obtaining the approximate MED AM_2 (Step ③). The same procedure is iteratively applied to subsequent segments, starting with S_3 , following the reuse or recompute path based on its STS with the previous segment.

B. Tunable-Grained Bucket Strategy

1) **Overview:** Fig. 4 illustrates the overview of a mechanism, which is designed with tunable granularity to meet

varying demands for accuracy and speed in different applications. This mechanism attempts to compute the approximate MED with the desired accuracy ϵ for a specific application. For a given application $\mathbb{S}$, we first execute the coarse-grained method, STREAM-CG. It returns an initial approximate MED, AM_1 , along with its achieved accuracy ϵ' . Based on the relationship between ϵ' and the desired accuracy ϵ , the following cases are considered:

- Case 1: If $\epsilon' \geq \epsilon$, AM_1 is directly used as final result, i.e., branch (a) in Fig. 4, ensuring both accuracy and speed.
- Case 2: If $\epsilon' < \epsilon$, it executes STREAM-FG, which narrows bucket boundaries to enhance accuracy and obtain a new approximate MED, AM_2 , i.e., branch (b) in Fig. 4. While STREAM-FG improves accuracy, it incurs additional computational cost.
- Case 3: If $\epsilon' = 1$, typically observed in highly concentrated data, the MED can be efficiently approximated by the mean (μ). In the extreme case where all data are identical, AM equals μ . Thus, μ can be directly used as the approximate MED, i.e., branch (c) in Fig. 4, bypassing further computation.

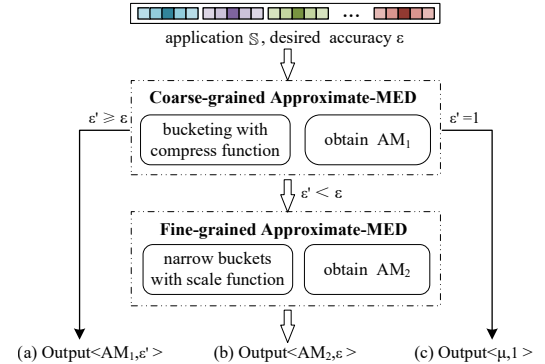


Fig. 4: The tunable mechanism for approximating MED with the desired accuracy ϵ , where ϵ' represents the achieved accuracy of AM_1 at the coarse-grained level.

2) **Coarse-grained Approximate MED:** We implement STREAM at the coarse-grained level (STREAM-CG), which involves computing bucket partitioning parameters, bucketing based on these parameters, and estimating AM using the bucket's boundaries. To efficiently represent data distribution, we adopt a synopsis structure $\mathbb{B} = \{B_{\frac{m-1}{2}-}, \dots, B_{1-}, B_{0-}, B_{0+}, B_{1+}, \dots, B_{\frac{m-1}{2}+}\}$, where m is the size of the synopsis, and each bucket B_i counts the number of data that fall within a specific boundary.

To determine bucket boundaries, we compute the mean (μ) and standard deviations (σ) of data, which is critical for effective bucket partitioning. The choice of μ and σ as partitioning parameters is guided by the 68-95-99.7 rule [18], which ensures $\mu \pm 3\sigma$ covers nearly all observed data. Moreover, in applications with STS, the consistency of data distributions between consecutive data segments results in stable μ and σ , making them ideal for defining bucket boundaries.

With μ and σ as inputs, we define bucket boundaries using the compress function $\gamma(x) := \frac{x-\mu}{\sigma}$, which allocates each

data to fixed-width buckets based on its deviation from μ . This method allows a constant synopsis size. For data with $\gamma(x)$ value outside the bucket range, we assign them to the leftmost or rightmost bucket, ensuring all data are included.

Finally, given that we keep the count of data within each bucket in synopsis $\mathbb{B}$, STREAM approximates the MED by finding the first bucket in $\mathbb{B}$, where the cumulative counts exceed 50% of the data. Within this target bucket, we calculate approximate MED by an interpolation formula [19]

$$AM = L + \frac{\ell/2 - \sum freq}{freq_{med}} width, \quad (1)$$

where L is the lower boundary of the bucket containing MED, ℓ is the total number of data in S_i , $\sum freq$ is the cumulative count of data in all buckets below the MED bucket, $freq_{med}$ is the count in the MED bucket, and $width$ is the width of the MED bucket (e.g., σ in STREAM-CG).

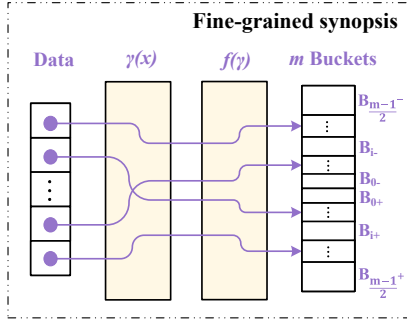


Fig. 5: The structure of STREAM-FG.

3) **Fine-grained Approximate MED:** While STREAM-CG efficiently computes the approximate MED with an achieved accuracy ϵ' , it may not meet the desired accuracy ϵ for certain applications. To address this limitation, STREAM-FG introduces a fine-grained bucket strategy to improve accuracy. A key feature of STREAM-FG is its use of the compression parameter γ , derived from the bucket partitioning parameters of STREAM-CG. The primary difference between STREAM-CG and STREAM-FG lies in how bucket boundaries are defined: STREAM-CG defines fixed-width bucket boundaries based on μ and σ , whereas STREAM-FG employs γ to refine bucket widths near the middle of the synopsis. Specifically, with γ as input, the bucket boundaries are determined by scale function $f(\gamma) = \omega \pi \tan^{-1}(2\gamma) + \delta$, where ω is the expansion coefficient, and δ is set to ensure the scale function can achieve the minimum and maximum values. This results in an uneven-width bucket structure, as shown in Fig. 5. Once the bucket partitioning parameter is determined, STREAM-FG follows the same steps as STREAM-CG for bucket allocation and MED calculation. The fine-grained bucket strategy of STREAM-FG significantly improves the accuracy of the approximate MED, making it well-suited to applications with high accuracy requirements.

4) **Theoretical Proof:** We conduct a theoretical proof on STREAM-CG, which proves our STREAM-CG can always work in initialize path and the rationality of STREAM-CG in reuse path.

Initialize Path. A bound of median location is measured by mean and standard deviation in initialize path, where the μ and σ are calculated on the samples where the median is estimated.

Proposition 1. Let X be a random variable on $\mathbb{R}$, θ be the corresponding distribution (an array of real values is a special case), let $\mu = \mathbb{E}[X]$ be the mean, and $\sigma = \sqrt{\mathbb{E}[(X - \mu)^2]}$ be the standard deviation, then the median m of X satisfies $|m - \mu| \leq \sigma$.

Proof. Let θ be the distribution of x in $\mathbb{R}$. We first show $\mathbb{E}|x - m| \leq \mathbb{E}|x - c|$ for any c . Without loss of generality, we assume $c > m$, then $\mathbb{E}|x - m| - \mathbb{E}|x - c| = \int_{-\infty}^m (m - x) - (c - x) d\theta(x) + \int_m^c (x - m) - (c - x) d\theta(x) + \int_c^{\infty} (x - m) - (x - c) d\theta(x) = (m - c)(\theta((-\infty, m]) - \theta([m, c]) - \theta([c, \infty))) - 2 \int_m^c (c - x) d\theta(x) \leq 0$. Further, by Jensen's inequality, $|\mu - m| = \mathbb{E}[|X - m|] \leq \mathbb{E}[|X - \mu|] \leq \sqrt{\mathbb{E}[(X - \mu)^2]} = \sigma$. $\square$

The previous proposition guarantees that our method always works in initialize path.

Reuse Path. Consider two random variables X_1 and X_2 , representing the distribution of values of two consecutive data segments in reuse path, and producing $\mu_1, \sigma_1, \mu_2, \sigma_2$ accordingly. The goal is to locate the median of X_2 among intervals generated by μ_1, σ_1 of X_1 . From the analysis of initialize path, it is known that $|m - \mu_2| < \sigma$. In the construction of the STREAM-CG algorithm, the threshold of path transfer is set to $3\sigma_1$.

Proposition 2. If $|\mu_1 - \mu_2| + \sigma_2 < 3\sigma_1$ holds, then the STREAM-CG stays in reuse path in estimating the current median m_2 .

Proof. From Proposition 1, we see that $|m_2 - \mu_2| < \sigma$. By the triangle inequality, $|m_2 - \mu_1| = |\mu_1 - \mu_2| + |m_2 - \mu_2| < |\mu_1 - \mu_2| + \sigma_2 < 3\sigma_1$ if the condition in this proposition holds. Then the STREAM stays in the reuse path for sure. $\square$

As a remark, Proposition 2 is a sufficient condition, meaning there could be cases not satisfying the inequality, but the reuse path still works.

The sufficient condition further transforms into $|\mu_1 - \mu_2| + (\sigma_2 - \sigma_1) < 2\sigma_1$, suggesting that when μ, σ of two consecutive data segments does not vary too much, and σ_1 is not too small to provide a large enough bar range, the STREAM stays in reuse path.

IV. EVALUATION AND ANALYSIS

A. Experimental Setup

1) **Platform and Setting:** All experiments are conducted on a server with 5.2GHz Intel(R) Core (TM) i9-12900K processors, and 32 GB of 5200MHz DDR5 RAM at OS kernel of Linux 5.15. We implement all codes with C and build them with gcc 7.5.0 (Ubuntu 7.5.0-6ubuntu2) and -O3 option. By default, the parameters of the comparing algorithms are set according to the recommendation of their authors.

TABLE II: Dataset description.

Dataset	Description	Dim	Type
<i>Miranda</i>	Turbulence modeling	$256 \times 384 \times 384$	double
<i>Hurricane</i>	Weather modeling	$500 \times 500 \times 100$	float
<i>NYX</i>	Cosmological simulation	$512 \times 512 \times 512$	float
<i>power</i>	Power measurements	1041120	float
<i>Moving MNIST</i>	Video	64×64	integer
<i>Lena</i>	Image	512×512	integer
<i>zipf</i>	Synthetic	10000	integer
<i>norm</i>	Synthetic	10000	float
<i>constant</i>	Synthetic	10000	integer

2) **Baselines**: To evaluate the effectiveness of our algorithms, we compare them against the existing exact method, EXACT-MED (based on QUICKSORT [20]), and SOTA approximate algorithms: C++ implementation of DDSketch³ and KLL [14]. We do not compare against HDRHistogram [13], as its C implementation is only applicable to positive integer data.

3) **Datasets**: Table II summarizes the 9 applications in our comparison experiments. The first six are real-world applications. Specifically, the *Miranda*, *Hurricane*, and *NYX* are produced by scientific applications from the Scientific Data Reduction Benchmarks suite¹. The *power*, used in [12], consists of Global Active Power measurements from the UCI Individual household electric power consumption dataset. The *Moving MNIST* and *Lena* are significant applications for computer version from Kaggle public dataset. The last three applications are synthetic, *zipf* consists of data generated from a zipfian distribution $zipf(1.2, 10000)$, *norm* consists of data generated from a normal distribution $\mathcal{N}(0, 1)$, and *constant* consists of data with the same constant value 1.

B. Comparison to Baselines

The time cost and accuracy results are summarized in Table III, with time measured in milliseconds. The accuracy rate of approximate MED relative to exact MED indicates that the relative error of approximate MED, i.e. $\frac{|MED - \hat{MED}|}{\max(x) - \min(x)}$, $x \in \mathbb{S}_i$, can be less than 5%.

1) **Time Cost**: We experiment ten times on each application and use the average run time as the result. As shown in Table III, STREAM-CG and STREAM-FG achieve significant speed improvements. Specifically, STREAM-CG outperforms EXACT-MED, DDSketch, and KLL up to 40.2x, 10.0x, and 71.2x (17.6x, 7.8x, and 15.8x on average), respectively. STREAM-FG outperforms EXACT-MED, DDSketch, and KLL up to 19.8x, 10.0x, and 71.2x (4.7x, 2.2x, and 7.0x on average), respectively.

STREAM-FG exhibits approximately 4-7x higher time costs than STREAM-CG across all applications (except *constant*). The result is expected, as STREAM-CG needs only a single bucketing calculation, while STREAM-FG requires the fine-grained buckets strategy through the scale function succeeding the compress function. The time costs of STREAM-CG and STREAM-FG are found to be comparable for *constant*. Since they directly return μ as their MED, bypassing further bucketing calculations.

³<https://github.com/tudor1805/sketches-cpp/>

TABLE III: Comparison to different baselines over various applications, where n is the size of the application and “Accu Rate” denotes the accuracy rate of approximate MED relative to exact MED. The parameters of STREAM-FG are set as $\omega = 5$, $\delta = 23$, resulting in the size of synopsis $m = 47$.

velocity (n=37.7M)			velocityz (n=37.7M)			viscosity (n=37.7M)		
Algorithm	Accu Rate	Time	Accu Rate	Time	Accu Rate	Time		
EXACT-MED	/	613.98	/	630.94	/	1222.37		
DDSketch	100%	504.75	100%	512.63	100%	577.20		
KLL	99.99%	560.83	99.99%	593.73	99.99%	821.03		
STREAM-CG (our)	92.78%	55.01	92.38%	62.74	89.95%	58.05		
STREAM-FG (our)	100%	246.23	100%	248.30	100%	251.14		
TC (n=25M)			U (n=25M)			PRECIP!g (n=25M)		
Algorithm	Accu Rate	Time	Accu Rate	Time	Accu Rate	Time		
EXACT-MED	/	260.93	/	307.50	/	320.79		
DDSketch	31.72%	266.79	99.99%	167.15	86.60%	268.58		
KLL	99.98%	448.02	99.99%	407.67	99.99%	467.07		
STREAM-CG (our)	91.10%	33.04	91.39%	33.82	94.02%	35.44		
STREAM-FG (our)	99.98%	167.15	99.99%	165.56	99.94%	158.44		
temperature (n=134M)			velocity_x (n=134M)			velocity_y (n=134M)		
Algorithm	Accu Rate	Time	Accu Rate	Time	Accu Rate	Time		
EXACT-MED	/	3624.06	/	2500.35	/	2581.12		
DDSketch	100%	1495.17	94.94%	1722.38	98.44%	1731.18		
KLL	99.99%	2594.92	99.99%	1971.93	99.99%	2019.34		
STREAM-CG (our)	99.91%	179.48	89.37%	183.13	89.32%	179.70		
STREAM-FG (our)	100%	935.72	100%	881.51	100%	877.19		
power (n=1041K)			MovingMNIST (n=7.2M)			lena (n=26.2M)		
Algorithm	Accu Rate	Time	Accu Rate	Time	Accu Rate	Time		
EXACT-MED	/	24.80	/	122.09	/	1026.41		
DDSketch	95%	13.03	100%	14.29	100%	263.46		
KLL	49.85%	23.30	99.95%	41.55	50%	480.05		
STREAM-CG (our)	62.74%	1.54	100%	12.82	100%	34.50		
STREAM-FG (our)	86.95%	7.56	100%	48.43	100%	186.38		
zipf (n=1M)			norm (n=10M)			constant (n=10K)		
Algorithm	Accu Rate	Time	Accu Rate	Time	Accu Rate	Time		
EXACT-MED	/	391.58	/	821.97	/	0.99		
DDSketch	100%	99.91	100%	134.08	100%	0.50		
KLL	90.91%	193.17	100%	188.03	100%	3.56		
STREAM-CG (our)	100%	12.28	100%	20.44	100%	0.05		
STREAM-FG (our)	100%	85.06	100%	81.30	100%	0.05		

For *TC*, STREAM-CG achieves optimal time and high accuracy, since it requires only single-pass scan and compress function γ , which ensures relatively even data allocation across buckets for this application. Moreover, our STREAM-FG further improves the accuracy. For *temperature*, STREAM-CG achieves superior performance in both speed and accuracy, while STREAM-FG exhibits poor time efficiency. If $\epsilon' \geq \epsilon$, STREAM-CG directly returns AM_1 , making further tuning with STREAM-FG unnecessary.

2) **Accuracy of Approximation**: As shown in Table III, STREAM-FG achieves high accuracy across applications while minimizing time costs. Notably, for the highly skewed application *zipf*, STREAM-FG can achieve an accuracy rate of 100%. For the extreme case *constant*, STREAM-FG returns μ as its MED directly, which is exactly the ground truth.

The accuracy rate of STREAM-CG is slightly lower for two main reasons: 1) While STS exists between data segments, the degree of data dispersion for each segment is relatively large (e.g., in *power*). This results in a significant deviation of the μ and σ from the MED, which in turn impacts the accuracy of approximate MED based bucket boundaries. 2) In some applications, data is highly concentrated with a few extreme values (e.g., *U* exhibits high frequency near zero alongside the distribution of non-near-zero values). This results in most data falling into the bucket closest to the mean, limiting the effectiveness of the compress function. This is also the primary reason why we adopt the fine-grained bucket strategy.

C. Evaluation of Proposed Techniques

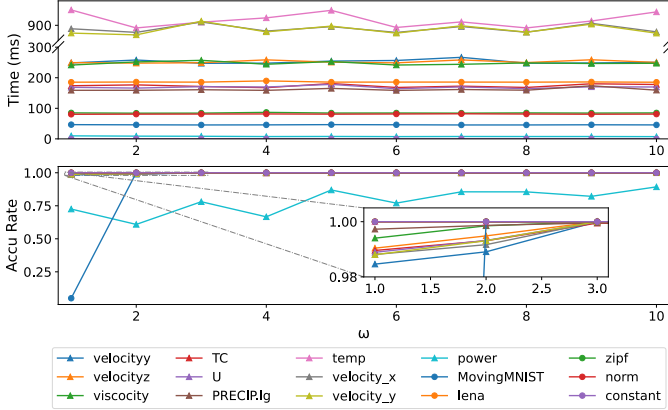
1) **Evaluating the Validity**: The validity refers to the effectiveness of our approximate MED methods to exploit

TABLE IV: Hit rate of bucket reuse based STS.

	velocityy	velocityz	viscosity	TC	U
STREAM-CG	100%	99.96%	99.99%	99.90%	99.92%
STREAM-FG	100%	99.98%	100%	99.94%	91.99%
	PRECIP.lg	temperature	velocity_x	velocity_y	power
STREAM-CG	99.76%	100%	100%	100%	81.67%
STREAM-FG	96.81%	100%	100%	100%	96.58%
	MovingMNIST	lena	zipf	norm	constant
STREAM-CG	100%	100%	100%	100%	NA
STREAM-FG	100%	100%	100%	100%	NA

the STS of the application. The validity is assessed using the *hitrate*, which reflects the proportion of correctly reused buckets between consecutive data segments. As shown in Table IV, *hitrate* exceeds 99% for all applications, except *power* using STREAM-CG and *constant*. For *power*, which represents power measurements recorded over a three-hour period. Due to daily fluctuations in energy consumption, the bucket widths in the STREAM-CG are not narrow enough to precisely capture slight deviations between consecutive segments. For *constant*, where the standard deviation is zero, no bucketing calculation is performed, and the mean is directly returned as the MED. This special case is not included in the standard statistical process. Thus, *hitrate* is marked as NA.

2) **Varying the Expansion Coefficient ω** : The expansion coefficient ω specifies the trade-off between the effectiveness and efficiency of STREAM-FG. We vary ω and apply STREAM-FG to each application, as shown in Fig. 6.

Fig. 6: Varying the expansion coefficient ω .

For higher concentrated applications, their σ are sufficiently small to return the MED directly, resulting in a near-constant and maximal precision. For other cases, sharp upward/downward trends are observed in Fig. 6. It is because when the dispersion of the data is large, the upper and lower bounds of the bucket vary due to changes in ω , which in turn fluctuates the accuracy. Moreover, while increasing ω to enhance accuracy, it simultaneously increases the number of buckets m (i.e., space cost) with a relatively stable time cost. Thereby, for concentrated applications, setting ω to an appropriate value can achieve better accuracy without significantly increasing the space cost.

V. CONCLUSION

In this paper, we propose STREAM, utilizing STS between data segments to reduce execution time and proving a tunable

granularity mechanism to meet the accuracy of application-specific requirements. Our theoretical analysis for correctness and efficiency proves the soundness of the proposed algorithms. The experimental results demonstrate the superiority of our algorithms, i.e., while maintaining nearly identical accuracy, STREAM outperforms the SOTA approximate methods DDSketch (up to 10x, 4.7x on average) and KLL (up to 71.2x, 10.1x on average).

ACKNOWLEDGMENT

This work was partially supported by the National Key R&D Program of China under Grant 2023YFB3001705, the National Natural Science Foundation of China (Grant No. 62321003 (Creative Research Groups), 62472160, and U21A20461), Hunan Province Graduate Research Innovation Project (CX20240425), and by the Ministry of Education, Singapore, under its Academic Research Fund Tier 1 (RG94/23).

REFERENCES

- [1] John Wellbelove, "Embed Template Library," <https://www.etlcpp.com/>, 2024.
- [2] "Software-Defined Radio Digital Signal Processing Library," <https://github.com/jgaeddert/liquid-dsp?tab=readme-ov-file>, 2024.
- [3] "FPM: A C++ header-only fixed-point math library," <https://github.com/MikeLankamp/fpm>, 2024.
- [4] "Eigen: a C++ template library for linear algebra," <https://eigen.tuxfamily.org/index.php>, 2024.
- [5] M. Liu, Z. Zhou, P. Shang, and D. Xu, "Fuzzified image enhancement for deep learning in iris recognition," *IEEE TFS*, vol. 28, no. 1, pp. 92–99, 2019.
- [6] S. Song, F. Gao, A. Zhang, J. Wang, and P. S. Yu, "Stream data cleaning under speed and acceleration constraints," *ACM TODS*, vol. 46, no. 3, pp. 1–44, 2021.
- [7] M. M. A. Patwary, S. Byna, N. R. Satish, N. Sundaram, Z. Lukić, V. Roytershteyn, M. J. Anderson, Y. Yao, Prabhat, and P. Dubey, "Bd-cats: big data clustering at trillion particle scale," in *SC*, 2015, pp. 1–12.
- [8] J. Han and M. Orshansky, "Approximate computing: An emerging paradigm for energy-efficient design," in *ETS*. IEEE, 2013, pp. 1–6.
- [9] K. Parasyris, G. Georgakoudis, H. Menon, J. Diffenderfer, I. Laguna, D. Osei-Kuffuor, and M. Schordan, "Hpac: evaluating approximate computing techniques on hpc openmp applications," in *SC*, 2021, pp. 1–14.
- [10] V. Gupta, T. Li, and P. Gupta, "Lac: Learned approximate computing," in *DATE*. IEEE, 2022, pp. 1169–1172.
- [11] V. Leon, M. A. Hanif, G. Armeniakos, X. Jiao, M. Shafique, K. Pekmetzi, and D. Soudris, "Approximate computing survey, part i: terminology and software & hardware approximation techniques," *arXiv preprint arXiv:2307.11124*, 2023.
- [12] C. Masson, J. E. Rim, and H. K. Lee, "Ddsketch: A fast and fully-mergeable quantile sketch with relative-error guarantees," *Proceedings of the VLDB Endowment*, vol. 12, no. 12, pp. 2195–2205, 2019.
- [13] Gil Tene, "HdrHistogram: A High Dynamic Range (HDR) Histogram," <https://github.com/HdrHistogram/HdrHistogram>, 2024.
- [14] "Apache DataSketches Core C++ Library," <https://github.com/apache/datasketches-cpp>, 2024.
- [15] F. Zhao, S. Maiyya, R. Wiener, D. Agrawal, and A. E. Abbadi, "KLL± approximate quantile sketches over dynamic datasets," *Proceedings of the VLDB Endowment*, vol. 14, no. 7, pp. 1215–1227, 2021.
- [16] Y. Zhang, X. Gao, Z. Chen, H. Zhong, H. Xie, and C. Yan, "Mining spatial-temporal similarity for visual tracking," *IEEE TIP*, vol. 29, pp. 8107–8119, 2020.
- [17] A. McGregor and P. Valiant, "The shifting sands algorithm," in *SODA*. SIAM, 2012, pp. 453–458.
- [18] V. Czitrom and P. D. Spagon, *Statistical case studies for industrial process improvement*. SIAM, 1997.
- [19] J. Han, J. Pei, and H. Tong, *Data mining: concepts and techniques*. Morgan kaufmann, 2022.
- [20] C. A. R. Hoare, "Algorithm 64: quicksort," *Communications of the ACM*, vol. 4, no. 7, p. 321, 1961.